

Decision Tree Classification – Airline Customer Satisfaction

Introduction

I am interested in predicting whether a future customer would be satisfied with the airline's services given customer feedback given previous customer feedback about their flight experience. I would like to construct and evaluate a model that can accomplish this goal. Specifically, I am interested in knowing which features are most important to customer satisfaction.

In this activity, I'll do the followings:

- Importing packages and loading data
- Exploring the data and completing the cleaning process
- Building a decision tree model
- Tuning hyperparameters using GridSearchCV
- Evaluating a decision tree model using a confusion matrix and various other plots

1: Imports ¶

Import packages

```
In [1]: 1 # Standard operational package imports
        2 import numpy as np
        3 import pandas as pd
        4
        5 # Important imports for modeling and evaluation
        6 from sklearn.model_selection import train_test_split
        7 from sklearn.model_selection import GridSearchCV
        8 from sklearn.tree import DecisionTreeClassifier
        9 from sklearn.tree import plot_tree
       10 import sklearn.metrics as metrics
       11
       12 # Visualization package imports
       13 import matplotlib.pyplot as plt
       14 import seaborn as sns
```

Load the dataset

```
In [2]: 1 # RUN THIS CELL TO IMPORT THE DATA.
        2
        3 file = "C:\\Users\\DajahV01\\Desktop\\DHF Assignment\\28. Decision Tree Model E
        4
        5 df_original = pd.read_csv(file)
```

Output the first 10 rows of data

```
In [3]: 1 df_original.head(n = 10)
```

Out[3]:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	0
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	0
5	satisfied	Loyal Customer	30	Personal Travel	Eco	1894	0	0	0
6	satisfied	Loyal Customer	66	Personal Travel	Eco	227	0	0	0
7	satisfied	Loyal Customer	10	Personal Travel	Eco	1812	0	0	0
8	satisfied	Loyal Customer	56	Personal Travel	Business	73	0	0	0
9	satisfied	Loyal Customer	22	Personal Travel	Eco	1556	0	0	0

10 rows × 22 columns



2: Data exploration, data cleaning, and model preparation

Prepare the data

Preparing the data includes:

- Exploring the data
- Checking for missing values
- Encoding the data
- Renaming a column
- Creating the training and testing data

Explore the data

```
In [4]: 1 df_original.dtypes
```

```
Out[4]: satisfaction          object
Customer Type                object
Age                          int64
Type of Travel               object
Class                        object
Flight Distance              int64
Seat comfort                  int64
Departure/Arrival time convenient int64
Food and drink                int64
Gate location                 int64
Inflight wifi service         int64
Inflight entertainment        int64
Online support                 int64
Ease of Online booking         int64
On-board service              int64
Leg room service              int64
Baggage handling              int64
Checkin service               int64
Cleanliness                   int64
Online boarding               int64
Departure Delay in Minutes     int64
Arrival Delay in Minutes       float64
dtype: object
```

Output unique values

The `Class` column is ordinal (meaning there is an inherent order that is significant). For example, airlines typically charge more for 'Business' than 'Eco Plus' and 'Eco'.

```
In [5]: 1 df_original["Class"].unique()
```

```
Out[5]: array(['Eco', 'Business', 'Eco Plus'], dtype=object)
```

Check the counts of the predicted labels

```
In [6]: 1 df_original['satisfaction'].value_counts(dropna = False)
```

```
Out[6]: satisfaction
satisfied      71087
dissatisfied    58793
Name: count, dtype: int64
```

Key insights:

- There are 71087 satisfied customers and 58793 dissatisfied customers.
- 54.7 percent (71087/129880) of customers were satisfied. This value can be compared to a decision tree's model accuracy.

Check for missing values

The sklearn decision tree implementation does not support missing values.

```
In [7]: 1 df_original.isnull().sum()
```

```
Out[7]: satisfaction                0  
Customer Type                    0  
Age                             0  
Type of Travel                  0  
Class                          0  
Flight Distance                 0  
Seat comfort                    0  
Departure/Arrival time convenient 0  
Food and drink                  0  
Gate location                   0  
Inflight wifi service           0  
Inflight entertainment          0  
Online support                  0  
Ease of Online booking          0  
On-board service                0  
Leg room service                0  
Baggage handling                0  
Checkin service                 0  
Cleanliness                     0  
Online boarding                 0  
Departure Delay in Minutes      0  
Arrival Delay in Minutes        393  
dtype: int64
```

Check the number of rows and columns in the dataset

```
In [8]: 1 df_original.shape
```

```
Out[8]: (129880, 22)
```

Drop the rows with missing values

```
In [9]: 1 df_subset = df_original.dropna(axis=0).reset_index(drop = True)
```


Check for missing values

```
In [10]: 1 df_subset.isna().sum()
```

```
Out[10]: satisfaction      0
Customer Type      0
Age      0
Type of Travel      0
Class      0
Flight Distance      0
Seat comfort      0
Departure/Arrival time convenient      0
Food and drink      0
Gate location      0
Inflight wifi service      0
Inflight entertainment      0
Online support      0
Ease of Online booking      0
On-board service      0
Leg room service      0
Baggage handling      0
Checkin service      0
Cleanliness      0
Online boarding      0
Departure Delay in Minutes      0
Arrival Delay in Minutes      0
dtype: int64
```

Check the number of rows and columns in the dataset again

```
In [11]: 1 df_subset.shape
```

```
Out[11]: (129487, 22)
```

Encode the data

Four columns (satisfaction , Customer Type , Type of Travel , Class) are the pandas dtype object. Decision trees need numeric columns. I convert the ordinal Class column into numeric.

```
In [12]: 1 df_subset['Class'] = df_subset['Class'].map({"Business": 3, "Eco Plus": 2, "Eco
```

Represent the data in the target variable numerically

```
In [13]: 1 df_subset['satisfaction'] = df_subset['satisfaction'].map({"satisfied": 1, "dis
```

Convert categorical columns into numeric

There are other columns in the dataset that are still categorical.

```
In [14]: 1 df_subset = pd.get_dummies(df_subset, drop_first = True)
```

Check column data types

I have converted categorical columns into numeric.

```
In [15]: 1 df_subset.dtypes
```

```
Out[15]: satisfaction      int64
Age                       int64
Class                     int64
Flight Distance           int64
Seat comfort              int64
Departure/Arrival time convenient int64
Food and drink            int64
Gate location             int64
Inflight wifi service     int64
Inflight entertainment    int64
Online support            int64
Ease of Online booking    int64
On-board service          int64
Leg room service          int64
Baggage handling          int64
Checkin service           int64
Cleanliness               int64
Online boarding           int64
Departure Delay in Minutes int64
Arrival Delay in Minutes  float64
Customer Type_disloyal Customer bool
Type of Travel_Personal Travel bool
dtype: object
```

Create the training and testing data

I put 75% of the data into a training set and the remaining 25% into a testing set.

```
In [16]: 1 y = df_subset["satisfaction"]
2
3 X = df_subset.copy()
4 X = X.drop("satisfaction", axis = 1)
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

3: Model building

Fit a decision tree classifier model to the data

```
In [17]: 1 decision_tree = DecisionTreeClassifier(random_state=0)
          2
          3 decision_tree.fit(X_train, y_train)
          4
          5 dt_pred = decision_tree.predict(X_test)
```

4: Results and evaluation

```
In [18]: 1 print("Decision Tree")
          2 print("Accuracy:", "%.6f" % metrics.accuracy_score(y_test, dt_pred))
          3 print("Precision:", "%.6f" % metrics.precision_score(y_test, dt_pred))
          4 print("Recall:", "%.6f" % metrics.recall_score(y_test, dt_pred))
          5 print("F1 Score:", "%.6f" % metrics.f1_score(y_test, dt_pred))
```

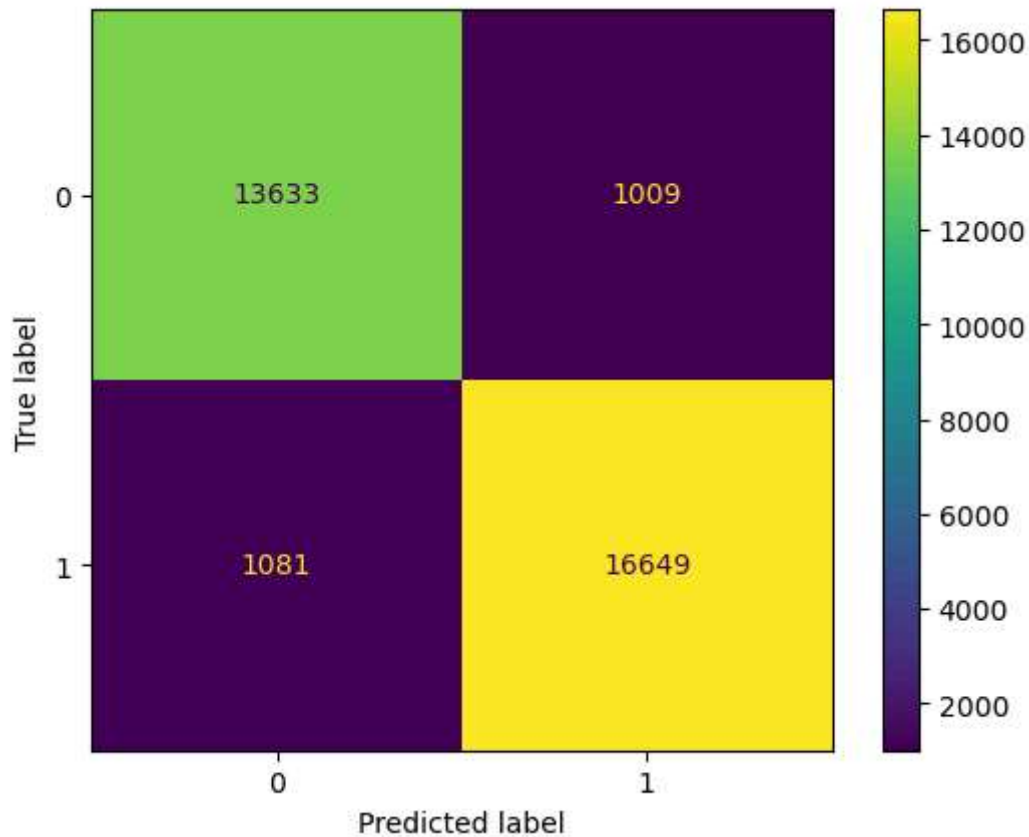
Decision Tree
Accuracy: 0.935438
Precision: 0.942859
Recall: 0.939030
F1 Score: 0.940940



Produce a confusion matrix

```
In [19]: 1 cm = metrics.confusion_matrix(y_test, dt_pred, labels = decision_tree.classes_)
          2 disp = metrics.ConfusionMatrixDisplay(confusion_matrix = cm, display_labels = de
          3 disp.plot()
```

Out[19]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x26808e84110> ⌵



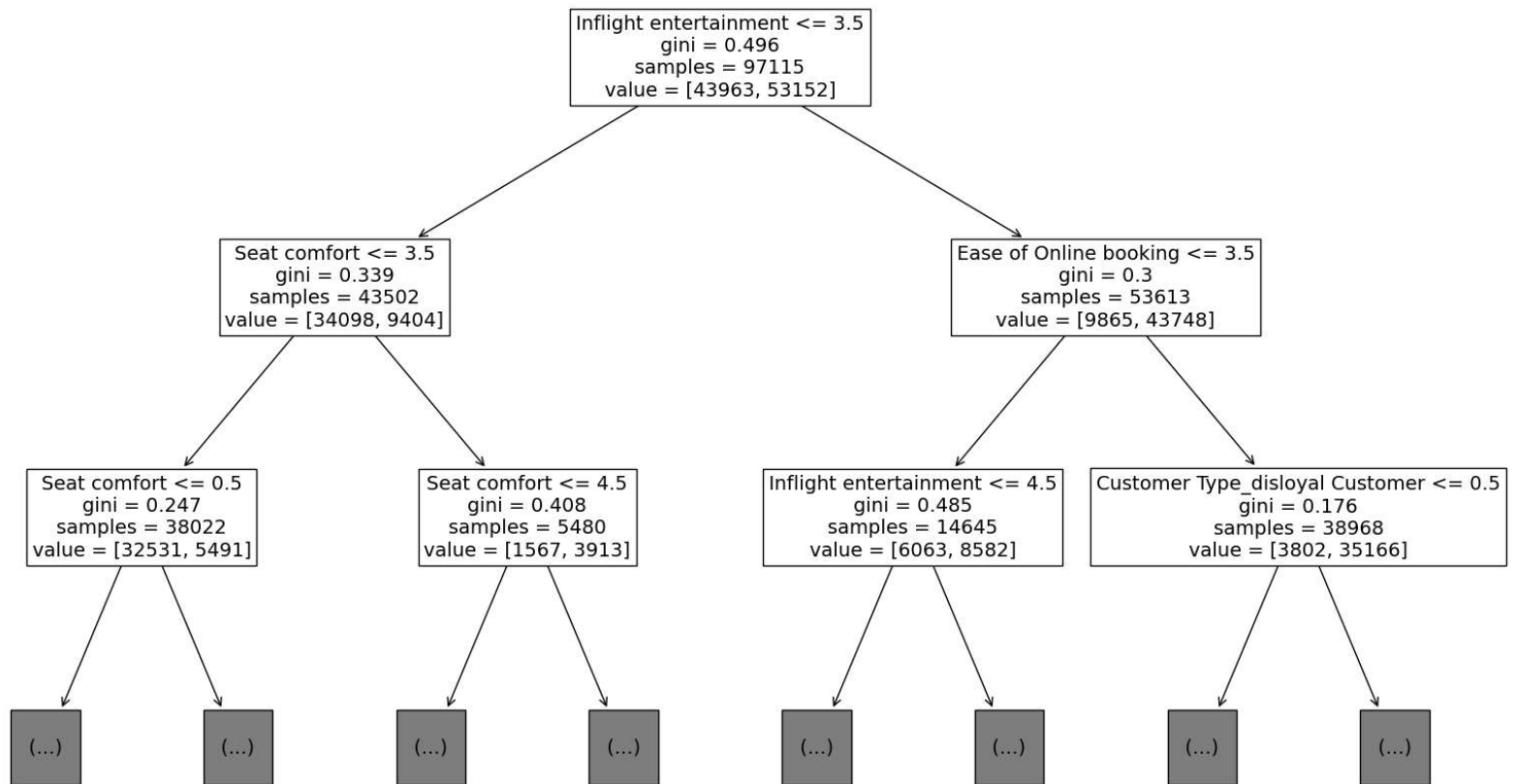
Key insights:

- In the confusion matrix, there are a high proportion of true positives and true negatives (where the matrix accurately predicted that the customer would be satisfied or dissatisfied, respectively).

- The matrix also had a relatively low number of false positives and false negatives (where the

Plot the decision tree

```
In [20]: 1 plt.figure(figsize=(20,12))  
2 plot_tree(decision_tree, max_depth=2, fontsize=14, feature_names=X.columns);
```



Calculate feature importance

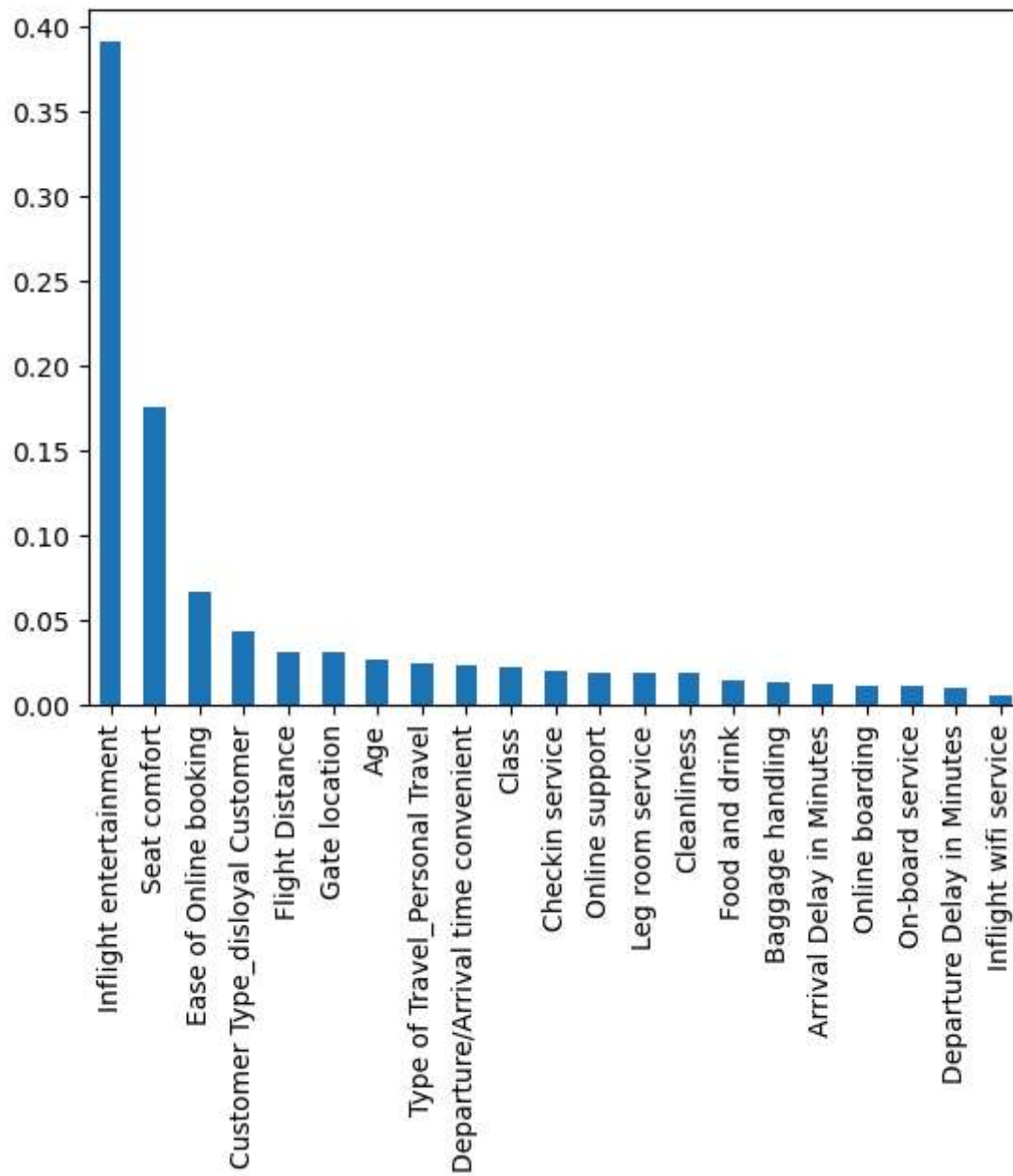
The `DecisionTreeClassifier` class comes with a `feature_importances_` (https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html#sklearn.tree.DecisionTreeClassifier.feature_importances_) attribute that provides access to the features' relative importance in the model. The feature importances are computed when the `fit()` method is called on the `DecisionTreeClassifier` instance. In other words, it's information that is generated during model training.

For each feature used in the tree, the algorithm finds all of the decision nodes that use that particular feature as the split criterion. Then for each of those decision nodes it computes the decrease in Gini impurity (or entropy, or log loss, or whatever metric you select when you fit the model—default is Gini impurity) that results from that split (so, the decrease from parent to children). Then the algorithm sums up the decreases across all the decisions made using that feature and expresses it as a percentage of the total decrease that resulted from *all* features.

Using the `feature_importances_` attribute to fetch the relative importances of each feature, I can then plot the results.



```
In [21]: 1 importances = decision_tree.feature_importances_  
2  
3 forest_importances = pd.Series(importances, index=X.columns).sort_values(ascending=True)  
4  
5 fig, ax = plt.subplots()  
6 forest_importances.plot.bar(ax=ax);
```

The feature importance graph seems to confirm that 'Inflight entertainment', 'Seat comfort', and 'Ease of Online booking' are the most important features for this model.

Hyperparameter tuning

In this section, I want to find the best values for the hyperparameters `max_depth` and `min_samples_leaf` using grid search and cross validation.

```
In [22]: 1 tree_para = {'max_depth':[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,30]
2          'min_samples_leaf': [2,3,4,5,6,7,8,9, 10, 15, 20, 50]}
3
4 scoring = {'accuracy', 'precision', 'recall', 'f1'}
```

Check combinations of values

```
In [23]: 1 tuned_decision_tree = DecisionTreeClassifier(random_state=0)
2
3 clf = GridSearchCV(tuned_decision_tree,
4                   tree_para,
5                   scoring = scoring,
6                   cv=5,
7                   refit="f1")
8
9 clf.fit(X_train, y_train)
```

```
Out[23]:  ▸ GridSearchCV
          ▸ estimator: DecisionTreeClassifier
            ▸ DecisionTreeClassifier
```

Compute the best combination of values for the hyperparameters

```
In [24]: 1 clf.best_estimator_
```

```
Out[24]: ▾ DecisionTreeClassifier
          DecisionTreeClassifier(max_depth=18, min_samples_leaf=2, random_state=0) ▾
```

Key insight:

- After running the DecisionTreeClassifier, the maximum depth is 18 and the minimum number of samples is two, meaning this is the best combination of values.

Determine the best average validation score

```
In [25]: 1 print("Best Avg. Validation Score: ", "%.4f" % clf.best_score_)
```

```
Best Avg. Validation Score:  0.9454
```

Key insight:

The best validation score is 0.9454.

Determine the "best" decision tree model's accuracy, precision, recall, and F1 score

```

In [26]: 1 results = pd.DataFrame(columns=['Model', 'F1', 'Recall', 'Precision', 'Accuracy']
2
3 def make_results(model_name, model_object):
4     """
5     Accepts as arguments a model name (my choice - string) and
6     a fit GridSearchCV model object.
7
8     Returns a pandas df with the F1, recall, precision, and accuracy scores
9     for the model with the best mean F1 score across all validation folds.
10    """
11
12    # Get all the results from the CV and put them in a df.
13    cv_results = pd.DataFrame(model_object.cv_results_)
14
15    # Isolate the row of the df with the max(mean f1 score).
16    best_estimator_results = cv_results.iloc[cv_results['mean_test_f1'].idxmax()]
17
18    # Extract accuracy, precision, recall, and f1 score from that row.
19    f1 = best_estimator_results.mean_test_f1
20    recall = best_estimator_results.mean_test_recall
21    precision = best_estimator_results.mean_test_precision
22    accuracy = best_estimator_results.mean_test_accuracy
23
24    # Create a table of results.
25    table = pd.DataFrame()
26    table = table._append({'Model': model_name,
27                          'F1': f1,
28                          'Recall': recall,
29                          'Precision': precision,
30                          'Accuracy': accuracy},
31                          ignore_index=True)
32
33    return table
34
35 result_table = make_results("Tuned Decision Tree", clf)
36
37 result_table

```

Out[26]:

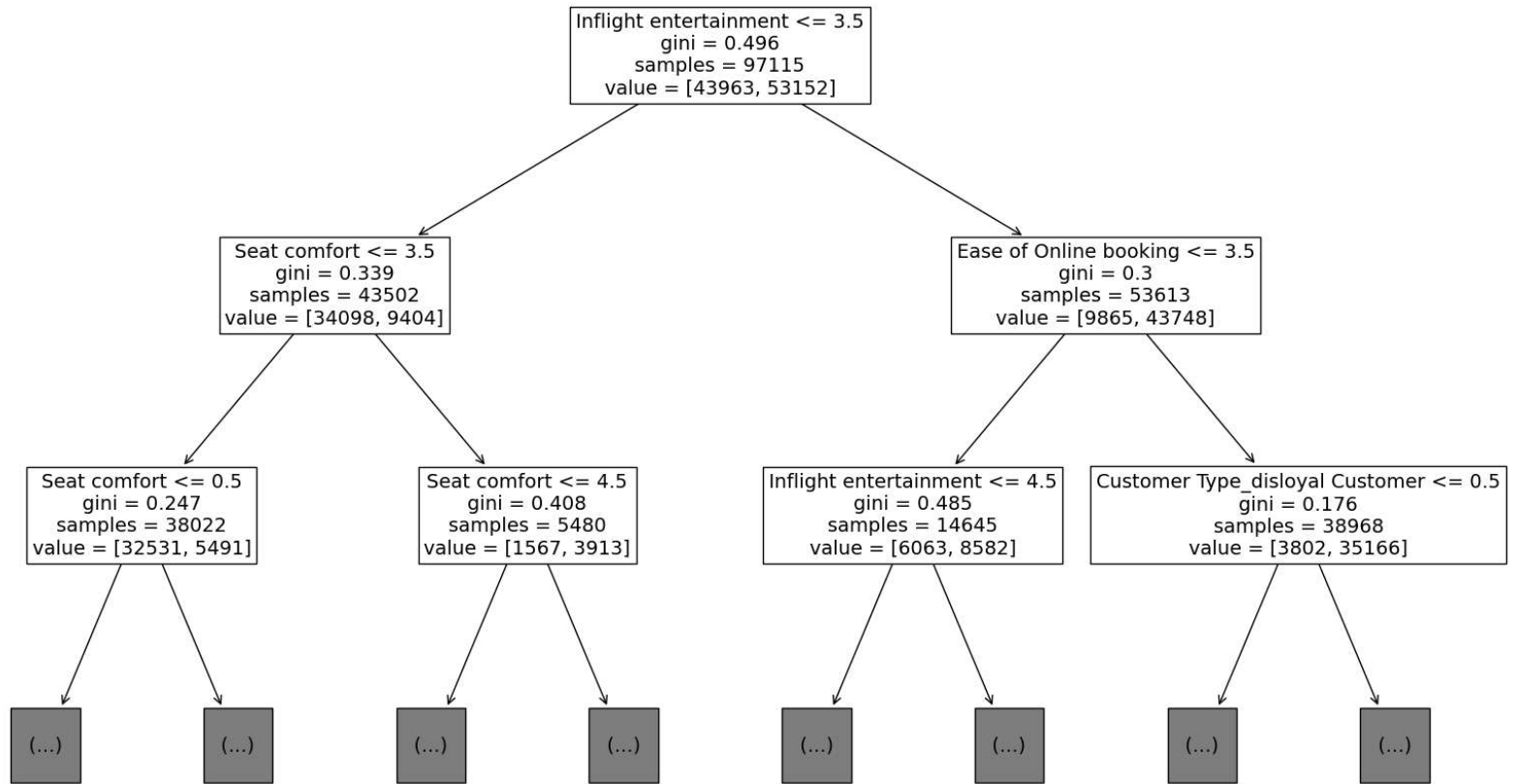
	Model	F1	Recall	Precision	Accuracy
0	Tuned Decision Tree	0.945422	0.935863	0.955197	0.940864

Key insight:

The F1 score for the decision tree that was not hyperparameter tuned is 0.940940 and the F1 score for the hyperparameter-tuned decision tree is 0.945422. While ensuring that overfitting doesn't occur is necessary for some models, it didn't make a meaningful difference in improving this model.

Plot the "best" decision tree

```
In [27]: 1 plt.figure(figsize=(20,12))  
2 plot_tree(clf.best_estimator_, max_depth=2, fontsize=14, feature_names=X.column
```

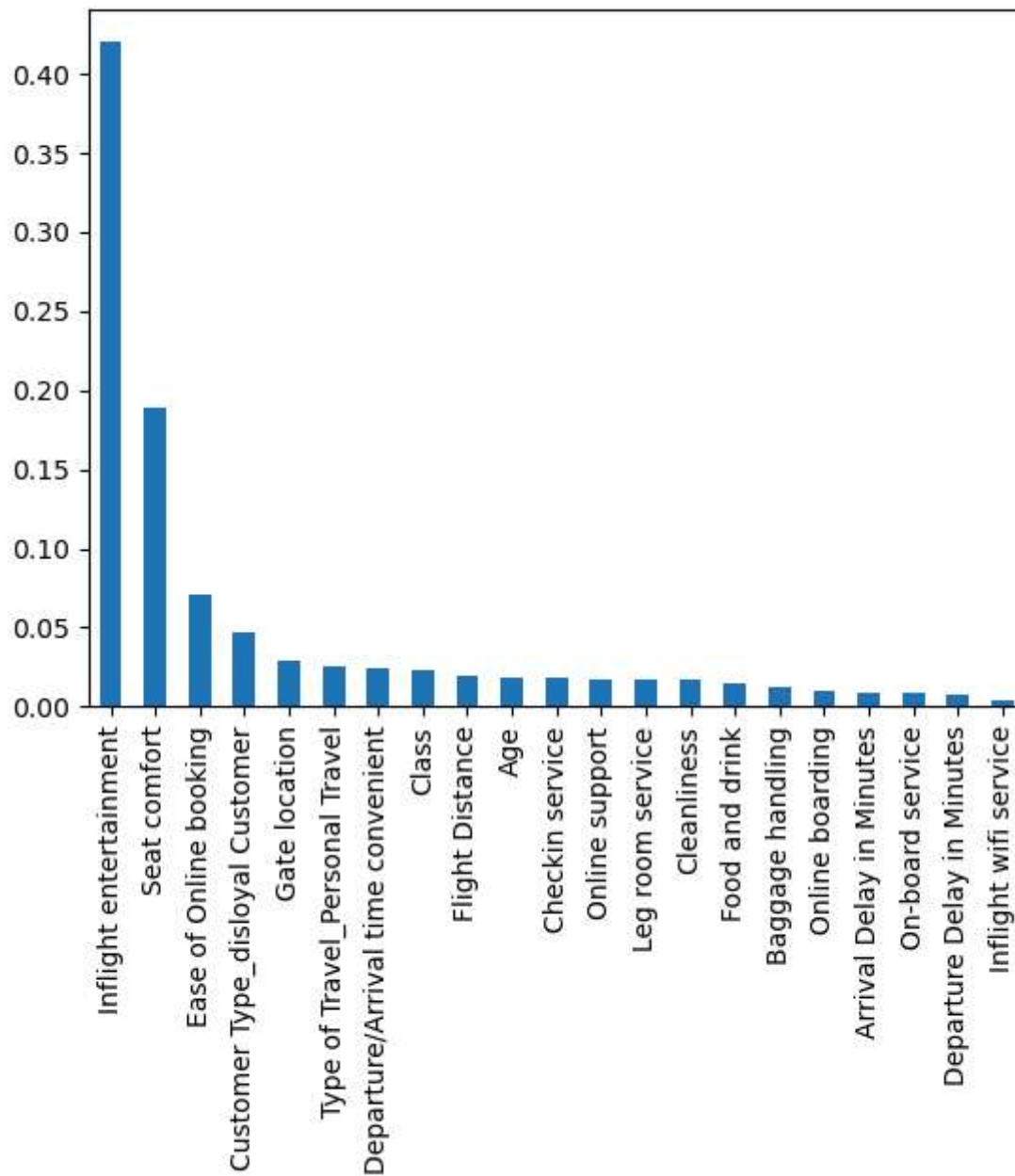


The plot makes it seem like 'Inflight entertainment' , 'Seat comfort' , and 'Ease of Online booking' are among the most important features. The code below outputs a "most important" features graph from the model.

Build another feature importance graph

I build another feature importance graph to validate which features are most important.

```
In [28]: 1 importances = clf.best_estimator_.feature_importances_  
2  
3 forest_importances = pd.Series(importances, index=X.columns).sort_values(ascending=True)  
4  
5 fig, ax = plt.subplots()  
6 forest_importances.plot.bar(ax=ax);
```

The feature importance graph seems to confirm that 'Inflight entertainment', 'Seat comfort', and 'Ease of Online booking' are the "most important" features for this model.

Considerations

My findings:

- Decision trees accurately predicted satisfaction over 94 percent of the time.
- The confusion matrix is useful as it shows a similar number of true positives and true negatives.
- The visualization of the decision tree and the feature importance graph both suggest that 'Inflight entertainment', 'Seat comfort', and 'Ease of Online booking' are the most important features in the model.

Recommendations:

- Customer satisfaction is highly tied to 'Inflight entertainment', 'Seat comfort', and 'Ease of Online booking'. Improving these experiences should lead to better customer satisfaction.
- The success of the model suggests that the airline should invest more effort into model building and model understanding since this model seemed to be very good at predicting customer satisfaction.